

LAPORAN TEAM BASED PROJECT

DISASTER RESPONSE LOGISTICS SYSTEM

Disusun untuk memenuhi tugas Mata Kuliah Algoritma dan Struktur Data



Dosen Pengampu:

Bapak Dr.Eng. Ir. Aji Ery Burhandenny ST., M.AIT.

Disusun oleh Kelompok 4:

1. Raihan Sajidan (25051030017)
2. Muhammad Ya'afi Daffa R. (25051030029)
3. Mohammad Nafif Bhanusetyo H. (25051030032)
4. Hafidh Aulia Dzakwan (25051030036)

PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA

2026

Kata Pengantar

Puji syukur kehadiran Allah SWT atas segala rahmat dan karunia-Nya sehingga laporan *Team Based Project* yang berjudul “Disaster Response Logistics System” ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas Mata Kuliah Algoritma dan Struktur Data. Dalam penyusunan proyek dan laporan ini, penulis memperoleh banyak pengetahuan serta pengalaman mengenai penerapan algoritma dan struktur data dalam pengembangan sistem logistik tanggap bencana. Proyek ini diharapkan dapat memberikan gambaran mengenai bagaimana teknologi dapat membantu proses distribusi bantuan secara lebih efektif dan efisien pada situasi bencana. Penulis menyadari bahwa laporan ini masih memiliki banyak kekurangan, baik dari segi penyusunan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu Mata Kuliah Algoritma dan Struktur Data serta seluruh pihak yang telah membantu dalam penyusunan laporan ini. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

Yogyakarta, 16 Mei 2026

Kelompok 4

DAFTAR ISI

Kata Pengantar.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Sistem	1
1.3 Batasan Implementasi	2
1.4 Diagram Arsitektur Sistem.....	2
BAB II PERANCANGAN SISTEM	3
2.1 Pemilihan Struktur Data.....	3
2.1.1 Graph.....	3
2.1.2 Priority Queue	3
2.1.3 Binary Search Tree (BST).....	3
2.1.4 Stack.....	4
2.2 Diagram Interaksi antar Modul.....	4
2.3 Pseudocode	5
BAB III IMPLEMENTASI.....	9
3.1 Implementasi Modul Sistem	9
3.1.1 Modul Binary Search Tree (BST).....	9
3.1.2 Modul Priority Queue	9
3.1.3 Modul Graph.....	9
3.1.4 Modul Stack	9
3.1.5 Implementasi Algoritma Dijkstra.....	9
3.1.6 Implementasi Command Line Interface (CLI)	10
3.2 Contoh Input dan Output Sistem.....	10
BAB IV ANALISIS Big-O	13
4.1 Analisis Kompleksitas.....	13
4.1.1 Tabel Ringkasan.....	13
4.2 Perbandingan Teoritis dengan Eksperimen.....	13
BAB V HASIL EKSPERIMEN	15
5.1 Tabel Runtime	15
5.2 Diskusi Hasil Prediksi Big-O.....	16

BAB VI JAWABAN PERTANYAAN ANALISIS	18
6.1 Pertanyaan Analisis	18
6.1.1 Analisis Priority Queue Bantuan.....	18
6.1.2 Analisis Perubahan Jalur pada Dijkstra.....	19
6.1.3 Analisis Multi-Source BFS	19
6.1.4 Analisis Stack Log Pengiriman.....	20
6.1.5 Analisis Sorting Laporan Bencana.....	21
BAB VII KESIMPULAN	23
7.1 Kesimpulan	23
7.2 Refleksi Trade-Off Desain	23
7.3 Saran Pengembangan	24
Lampiran	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia merupakan salah satu negara dengan tingkat risiko bencana alam yang tinggi karena letaknya berada pada pertemuan tiga lempeng tektonik utama. Kondisi tersebut menyebabkan bencana seperti gempa bumi, banjir, tanah longsor, dan erupsi gunung berapi sering terjadi. Dalam situasi bencana, proses distribusi bantuan logistik menjadi hal yang sangat penting karena keterlambatan bantuan dapat memperburuk kondisi masyarakat terdampak.

Untuk mendukung proses penanganan bencana yang lebih efektif, dibutuhkan sistem logistik yang mampu mengatur prioritas bantuan, menentukan rute distribusi yang efisien, serta melakukan pengelolaan data secara terstruktur. Perkembangan teknologi memungkinkan penerapan algoritma dan struktur data untuk membantu pengelolaan sistem logistik tanggap bencana.

Pada proyek ini digunakan beberapa struktur data dan algoritma, yaitu Priority Queue untuk menentukan prioritas bantuan, algoritma Dijkstra untuk mencari rute distribusi tercepat, Binary Search Tree (BST) untuk pengelolaan data lokasi bencana, serta Stack untuk menyimpan riwayat pengiriman bantuan. Sistem dirancang untuk mensimulasikan distribusi bantuan pada 35 lokasi terdampak dengan dukungan 3 depot logistik utama.

Melalui penerapan algoritma dan struktur data tersebut, proyek ini diharapkan dapat memberikan simulasi sistem logistik tanggap bencana yang lebih efektif sekaligus menjadi implementasi konsep pada Mata Kuliah Algoritma dan Struktur Data.

1.2 Tujuan Sistem

Tujuan dari pembuatan *Disaster Response Logistics System* ini adalah sebagai berikut:

1. Mengimplementasikan konsep algoritma dan struktur data ke dalam sistem logistik tanggap bencana.
2. Membantu menentukan prioritas pengiriman bantuan ke lokasi yang paling kritis menggunakan *Priority Queue*.
3. Menentukan rute distribusi bantuan yang optimal menggunakan algoritma Dijkstra.
4. Mempermudah proses registrasi, pencarian, dan pembaruan data lokasi bencana menggunakan *Binary Search Tree* (BST).
5. Menyimpan riwayat distribusi bantuan sebagai bentuk audit pengiriman menggunakan *Stack*.
6. Mensimulasikan sistem logistik bencana yang efisien dan terstruktur untuk mendukung

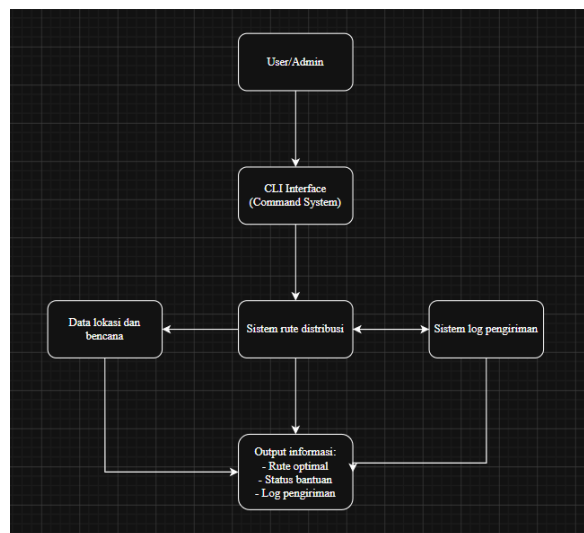
proses penanganan bencana.

1.3 Batasan Implementasi

Agar pembahasan dan implementasi sistem lebih terarah, maka ditetapkan beberapa batasan sebagai berikut:

1. Sistem hanya berupa simulasi logistik tanggap bencana dan tidak terhubung dengan data bencana secara real-time.
2. Jumlah lokasi terdampak yang digunakan dalam simulasi sebanyak 35 lokasi.
3. Sistem menggunakan 3 depot logistik sebagai pusat distribusi bantuan.
4. Jenis bantuan yang didistribusikan tidak dibedakan berdasarkan kategori tertentu.
5. Algoritma pencarian rute yang digunakan hanya algoritma Dijkstra.
6. Struktur data yang digunakan pada sistem meliputi *Priority Queue*, *Binary Search Tree* (BST), dan *Stack*.
7. Sistem dijalankan melalui antarmuka berbasis terminal atau *command line*.

1.4 Diagram Arsitektur Sistem



Gambar 1.1 Diagram Arsitektur Sistem

Diagram arsitektur sistem menggambarkan hubungan antara pengguna dengan komponen utama pada *Disaster Response Logistics System*. Pengguna berinteraksi dengan sistem melalui *Command Line Interface* (CLI) untuk menjalankan berbagai perintah seperti pengiriman bantuan, pencarian rute optimal, pembaruan level bencana, dan penampilan laporan logistik. Sistem kemudian memproses data lokasi bencana, menentukan jalur distribusi bantuan, serta mencatat aktivitas pengiriman. Hasil pemrosesan ditampilkan dalam bentuk informasi rute optimal, status bantuan, dan log pengiriman sehingga pengguna dapat memantau proses distribusi bantuan secara lebih terstruktur.

BAB II

PERANCANGAN SISTEM

2.1 Pemilihan Struktur Data

Dalam pengembangan *Disaster Response Logistics System*, pemilihan struktur data menjadi bagian penting untuk mendukung efisiensi proses distribusi bantuan bencana. Setiap struktur data dipilih berdasarkan fungsi dan kebutuhan sistem, seperti pengelolaan data lokasi, pengaturan prioritas bantuan, pencarian jalur distribusi, serta pencatatan aktivitas logistik. Struktur data yang digunakan pada sistem ini meliputi Graph, Priority Queue, Binary Search Tree (BST), dan Stack.

2.1.1 Graph

Graph digunakan untuk merepresentasikan jaringan jalan antara depot logistik dan lokasi bencana. Pada sistem ini, setiap lokasi direpresentasikan sebagai node, sedangkan hubungan jalan antar lokasi direpresentasikan sebagai edge berbobot jarak dan kapasitas distribusi. Struktur data graph dipilih karena mampu memodelkan hubungan antar lokasi secara fleksibel serta mendukung proses pencarian rute optimal menggunakan algoritma BFS dan Dijkstra. Implementasi graph pada sistem menggunakan adjacency list berbasis linked list agar penggunaan memori lebih efisien.

2.1.2 Priority Queue

Priority Queue digunakan untuk mengatur antrean bantuan berdasarkan tingkat prioritas bencana. Lokasi dengan level KRITIS diproses lebih dahulu dibanding lokasi dengan level SEDANG dan RINGAN. Struktur data ini dipilih karena sesuai dengan kebutuhan sistem tanggap darurat yang memerlukan mekanisme pemrosesan bantuan berdasarkan tingkat urgensi. Priority Queue pada sistem diimplementasikan menggunakan linked list terurut sehingga proses pengambilan bantuan prioritas tertinggi dapat dilakukan dengan cepat.

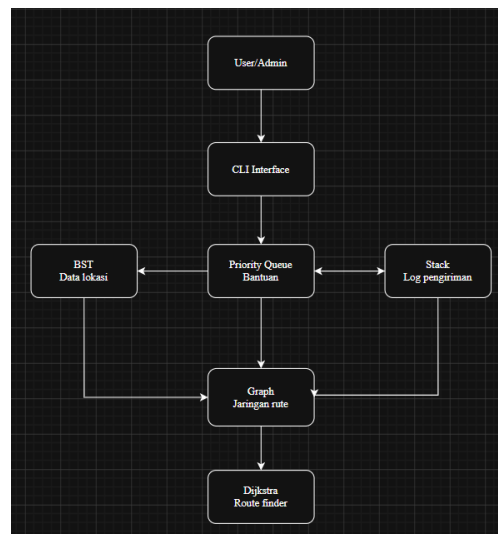
2.1.3 Binary Search Tree (BST)

Binary Search Tree (BST) digunakan untuk menyimpan data lokasi bencana dan depot logistik berdasarkan kode lokasi. BST dipilih karena mendukung proses pencarian, penyisipan, dan pembaruan data secara efisien. Selain itu, BST memungkinkan data lokasi ditampilkan secara terurut melalui traversal inorder. Pada sistem ini, setiap node BST menyimpan informasi berupa kode lokasi, nama lokasi, level bencana, jumlah populasi, dan status bantuan.

2.1.4 Stack

Stack digunakan untuk menyimpan log pengiriman bantuan dengan prinsip *Last In First Out* (LIFO). Struktur data ini dipilih karena sesuai untuk fitur rollback pengiriman, yaitu membatalkan pengiriman terakhir apabila terjadi kesalahan distribusi. Implementasi stack menggunakan linked list sehingga operasi push dan pop dapat dilakukan dengan kompleksitas $O(1)$. Selain itu, stack juga mempermudah proses pencatatan histori aktivitas logistik pada sistem.

2.2 Diagram Interaksi antar Modul



Gambar 2.1 Diagram Interaksi antar Modul

Diagram interaksi antar modul pada *Disaster Response Logistics System* menggambarkan hubungan antar komponen utama dalam proses distribusi bantuan bencana. Interaksi sistem dimulai dari pengguna melalui *Command Line Interface* (CLI) yang berfungsi sebagai pusat pengendali seluruh fitur sistem. Melalui CLI, pengguna dapat menjalankan perintah seperti pengiriman bantuan, pembaruan level bencana, pencarian rute optimal, dan penampilan log pengiriman.

Data lokasi bencana dan depot logistik dikelola menggunakan modul Binary Search Tree (BST) untuk mempermudah proses pencarian dan pembaruan data lokasi. Selanjutnya, data bantuan diproses menggunakan Priority Queue agar lokasi dengan tingkat bencana paling kritis dapat diprioritaskan terlebih dahulu. Sistem kemudian menggunakan Graph untuk merepresentasikan jaringan distribusi antar lokasi dan algoritma Dijkstra untuk menentukan jalur pengiriman tercepat dari depot menuju lokasi tujuan.

Selain itu, sistem juga menggunakan Stack untuk menyimpan riwayat pengiriman bantuan. Modul ini memungkinkan sistem melakukan pencatatan aktivitas logistik serta mendukung fitur rollback apabila terjadi kesalahan pengiriman. Dengan interaksi antar modul tersebut, sistem dapat menjalankan simulasi distribusi bantuan secara lebih terstruktur dan efisien.

2.3 Pseudocode

1. Generate Peta Bencana

FUNCTION generate_peta_bencana()

 buat beberapa depot

 buat beberapa lokasi bencana secara acak

 buat koneksi/rute antar lokasi

 RETURN daftar lokasi dan daftar rute

2. Insert Lokasi ke BST

FUNCTION insert_lokasi(lokalasi)

 IF tree kosong THEN

 jadikan lokasi sebagai root

 ELSE

 bandingkan kode lokasi

 simpan ke subtree kiri atau kanan

3. Search Lokasi pada BST

FUNCTION search_lokasi(kode)

 mulai dari root

 WHILE node masih ada DO

 IF kode ditemukan THEN

 RETURN lokasi

 ELSE IF kode lebih kecil THEN

 pindah ke kiri

 ELSE

 pindah ke kanan

 RETURN None

4. Enqueue Bantuan (Priority Queue)

FUNCTION enqueue_bantuan(bantuan)

 buat node baru

 IF queue kosong

 ATAU prioritas lebih tinggi THEN

 simpan di depan queue

 ELSE

 telusuri queue

 sisipkan sesuai urutan prioritas

5. Dequeue Bantuan

FUNCTION dequeue_bantuan()

 IF queue kosong THEN

 RETURN None

 ambil data head

 head pindah ke node berikutnya

 RETURN bantuan

6. Push Log Pengiriman ke Stack

FUNCTION push_log(data)

 buat node baru

 node.next = top

 top = node

7. BFS untuk Mencari Lokasi Terjangkau

FUNCTION bfs_akses(depot)

 visited = kosong

 masukkan depot ke queue

 WHILE queue tidak kosong DO

 ambil node dari queue

 telusuri semua tetangga

 IF belum dikunjungi THEN

tandai visited

masukkan ke queue

RETURN visited

8. Dijkstra untuk Rute Optimal

FUNCTION dijkstra(graph, depot)

set semua jarak = tak hingga

jarak depot = 0

WHILE masih ada node belum diproses DO

 pilih node dengan jarak minimum

 tandai sebagai visited

 telusuri semua tetangga

 hitung jarak baru

 IF jarak lebih kecil THEN

 update jarak

 simpan parent

RETURN dist dan parent

9. Rekonstruksi Jalur

FUNCTION reconstruct_path(parent, tujuan)

path = kosong

WHILE tujuan masih ada DO

 masukkan tujuan ke path

 tujuan = parent[tujuan]

balik urutan path

RETURN path

10. Main Program

PROGRAM utama

 buat BST lokasi

 buat graph rute

 buat priority queue bantuan

buat stack log pengiriman

generate data lokasi dan rute

masukkan data ke BST dan graph

WHILE program berjalan DO

 baca command user

 IF command = KIRIM THEN

 tambah bantuan ke antrian

 ELSE IF command = PROSES_BANTUAN THEN

 proses bantuan prioritas tertinggi

 ELSE IF command = RUTE_OPTIMAL THEN

 cari jalur terpendek dengan Dijkstra

 ELSE IF command = UPDATE_LEVEL THEN

 ubah level bencana

 ELSE IF command = TIDAK_TERJANGKAU THEN

 cari node tidak terjangkau dengan BFS

 ELSE IF command = LOG_PENGIRIMAN THEN

 tampilkan riwayat pengiriman

 ELSE IF command = LAPORAN_BENCANA THEN

 tampilkan seluruh lokasi

 ELSE IF command = KELUAR THEN

 hentikan program

BAB III

IMPLEMENTASI

3.1 Implementasi Modul Sistem

3.1.1 Modul Binary Search Tree (BST)

Modul Binary Search Tree (BST) digunakan untuk menyimpan dan mengelola data lokasi bencana serta depot logistik. Modul ini mendukung proses pencarian, penyisipan, dan pembaruan data lokasi berdasarkan kode lokasi secara efisien. Implementasi BST dilakukan menggunakan class `BSTNodeLok` dan `BSTLokasi` dengan operasi utama berupa `insert()`, `search()`, dan `update_level()`.

3.1.2 Modul Priority Queue

Modul Priority Queue digunakan untuk mengatur antrean bantuan berdasarkan tingkat prioritas bencana. Lokasi dengan level KRITIS diproses lebih dahulu dibanding level lainnya. Modul ini diimplementasikan menggunakan linked list terurut sehingga proses pengambilan bantuan prioritas tertinggi dapat dilakukan dengan cepat melalui operasi `enqueue()` dan `dequeue()`.

3.1.3 Modul Graph

Modul Graph digunakan untuk merepresentasikan jaringan distribusi antara depot logistik dan lokasi bencana. Setiap node merepresentasikan lokasi, sedangkan edge merepresentasikan hubungan jalan dengan bobot berupa jarak dan kapasitas distribusi. Modul ini mendukung proses pencarian rute optimal menggunakan algoritma BFS dan Dijkstra.

3.1.4 Modul Stack

Modul Stack digunakan untuk menyimpan log pengiriman bantuan menggunakan prinsip *Last In First Out* (LIFO). Modul ini berfungsi untuk mencatat histori distribusi bantuan serta mendukung fitur rollback apabila terjadi kesalahan pengiriman. Implementasi stack dilakukan menggunakan linked list dengan operasi utama `push()` dan `pop()`.

3.1.5 Implementasi Algoritma Dijkstra

Algoritma Dijkstra digunakan untuk menentukan jalur distribusi tercepat dari depot menuju lokasi tujuan berdasarkan bobot jarak pada graph. Algoritma bekerja dengan mencari node dengan jarak minimum yang belum dikunjungi, kemudian memperbarui jarak seluruh node tetangganya hingga diperoleh rute optimal distribusi bantuan.

3.1.6 Implementasi Command Line Interface (CLI)

Sistem *Disaster Response Logistics System* dijalankan menggunakan *Command Line Interface* (CLI) sebagai media interaksi antara pengguna dengan program. Melalui CLI, pengguna dapat menjalankan berbagai perintah seperti pengiriman bantuan, pencarian rute optimal, pembaruan level bencana, penampilan log pengiriman, serta pengecekan lokasi yang dapat dijangkau dari depot tertentu. Implementasi CLI dilakukan pada fungsi `main()` yang berperan sebagai pusat pengendali seluruh modul dan alur sistem.

```
Disaster Response Logistics System | Ketik BANTUAN untuk daftar perintah

>> BANTUAN

Perintah:
KIRIM <depot> <lokasi> <jenis> <jumlah>
PROSES_BANTUAN
RUTE_OPTIMAL <depot> <tujuan>
UPDATE_LEVEL <kode> <level>
TIDAK_TERJANGKAU <depot>
LOG_PENGIRIMAN
LAPORAN_BENCANA
KELUAR

>> |
```

Gambar 3.1 Simulasi CLI Bekerja

3.2 Contoh Input dan Output Sistem

Tabel 3.1 Contoh-Contoh Input dan Output

No	Deskripsi	Gambar
1	Contoh Input Output Laporan Bencana	<pre>>> LAPORAN_BENCANA Daftar Lokasi: DEPOT_0 Gudang Logistik 0 Level:3 Populasi:0 DEPOT_1 Gudang Logistik 1 Level:3 Populasi:0 DEPOT_2 Gudang Logistik 2 Level:3 Populasi:0 L000 Desa/Kelurahan-0 Level:3 Populasi:2764 L001 Desa/Kelurahan-1 Level:2 Populasi:3792 L002 Desa/Kelurahan-2 Level:1 Populasi:485 L003 Desa/Kelurahan-3 Level:2 Populasi:4837 L004 Desa/Kelurahan-4 Level:2 Populasi:2196 L005 Desa/Kelurahan-5 Level:2 Populasi:4303 L006 Desa/Kelurahan-6 Level:1 Populasi:1533 L007 Desa/Kelurahan-7 Level:3 Populasi:3777 L008 Desa/Kelurahan-8 Level:2 Populasi:4154 L009 Desa/Kelurahan-9 Level:3 Populasi:4126 L010 Desa/Kelurahan-10 Level:2 Populasi:1513 L011 Desa/Kelurahan-11 Level:3 Populasi:1620</pre>
2	Contoh Input Output Update Level Bencana	<pre>>> UPDATE_LEVEL L001 1 Level berhasil diperbarui. L001 : SEDANG -> KRITIS >> UPDATE_LEVEL L007 2 Level berhasil diperbarui. L007 : RINGAN -> SEDANG >> LAPORAN_BENCANA Daftar Lokasi: DEPOT_0 Gudang Logistik 0 Level:3 Populasi:0 DEPOT_1 Gudang Logistik 1 Level:3 Populasi:0 DEPOT_2 Gudang Logistik 2 Level:3 Populasi:0 L000 Desa/Kelurahan-0 Level:3 Populasi:2764 L001 Desa/Kelurahan-1 Level:1 Populasi:3792 L002 Desa/Kelurahan-2 Level:1 Populasi:485 L003 Desa/Kelurahan-3 Level:2 Populasi:4837 L004 Desa/Kelurahan-4 Level:2 Populasi:2196 L005 Desa/Kelurahan-5 Level:2 Populasi:4303 L006 Desa/Kelurahan-6 Level:1 Populasi:1533 L007 Desa/Kelurahan-7 Level:2 Populasi:3777 L008 Desa/Kelurahan-8 Level:2 Populasi:4154 L009 Desa/Kelurahan-9 Level:3 Populasi:4126 L010 Desa/Kelurahan-10 Level:2 Populasi:1513 L011 Desa/Kelurahan-11 Level:3 Populasi:1620</pre>

3	Contoh Input Output Pencarian Rute Optimal	<pre> >> RUTE_OPTIMAL DEPOT_0 L001 Rute Optimal: DEPOT_0 -> L018 -> L001 Total jarak: 93 km >> RUTE_OPTIMAL DEPOT_1 L001 Rute Optimal: DEPOT_1 -> L014 -> L031 -> L032 -> L001 Total jarak: 159 km >> RUTE_OPTIMAL DEPOT_2 L002 Rute Optimal: DEPOT_2 -> L015 -> L020 -> L000 -> L002 Total jarak: 249 km >> </pre>
4	Contoh Input Output Pengiriman Bantuan	<pre> >> KIRIM DEPOT_0 L000 MAKANAN 200 Bantuan masuk antrian prioritas. Antrian 1: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> KIRIM DEPOT_1 L005 TENDA 160 Bantuan masuk antrian prioritas. Antrian 1: 160 TENDA dari DEPOT_1 ke L005. Level SEDANG Antrian 2: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> KIRIM DEPOT_2 L009 AIR 450 Bantuan masuk antrian prioritas. Antrian 1: 160 TENDA dari DEPOT_1 ke L005. Level SEDANG Antrian 2: 450 AIR dari DEPOT_2 ke L009. Level SEDANG Antrian 3: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> KIRIM DEPOT_0 L001 OBAT 530 Bantuan masuk antrian prioritas. Antrian 1: 530 OBAT dari DEPOT_0 ke L001. Level KRITIS Antrian 2: 160 TENDA dari DEPOT_1 ke L005. Level SEDANG Antrian 3: 450 AIR dari DEPOT_2 ke L009. Level SEDANG Antrian 4: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> </pre>
5	Contoh Input Output Memproses Pengiriman Bantuan	<pre> >> PROSES_BANTUAN Bantuan diproses: Pengirim 530 OBAT dari DEPOT_0 ke L001 Antrian tersisa: Antrian 1: 160 TENDA dari DEPOT_1 ke L005. Level SEDANG Antrian 2: 450 AIR dari DEPOT_2 ke L009. Level SEDANG Antrian 3: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> PROSES_BANTUAN Bantuan diproses: Pengirim 160 TENDA dari DEPOT_1 ke L005 Antrian tersisa: Antrian 1: 450 AIR dari DEPOT_2 ke L009. Level SEDANG Antrian 2: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> PROSES_BANTUAN Bantuan diproses: Pengirim 450 AIR dari DEPOT_2 ke L009 Antrian tersisa: Antrian 1: 200 MAKANAN dari DEPOT_0 ke L000. Level RINGAN >> PROSES_BANTUAN Bantuan diproses: Pengirim 200 MAKANAN dari DEPOT_0 ke L000 Antrian tersisa: Antrian kosong! >> </pre>
6	Contoh Input Output Log Pengiriman	<pre> >> LOG_PENGIRIMAN Riwayat Pengiriman: 1 : 530 OBAT dari DEPOT_0 ke L001 2 : 160 TENDA dari DEPOT_1 ke L005 3 : 450 AIR dari DEPOT_2 ke L009 4 : 200 MAKANAN dari DEPOT_0 ke L000 >> </pre>

7	Contoh Input Output Daerah Tidak Terjangkau	<pre> >> TIDAK_TERJANGKAU DEPOT_0 Lokasi tidak terjangkau: Tidak ada >> TIDAK_TERJANGKAU DEPOT_1 Lokasi tidak terjangkau: Tidak ada >> TIDAK_TERJANGKAU DEPOT_2 Lokasi tidak terjangkau: Tidak ada >> </pre>
---	---	--

BAB IV

ANALISIS Big-O

4.1 Analisis Kompleksitas

Pada sistem *Disaster Response Logistics System*, digunakan beberapa struktur data dan algoritma seperti *Binary Search Tree* (BST), *Priority Queue*, *Stack*, *Graph*, *Breadth First Search* (BFS), dan algoritma Dijkstra. Setiap struktur data memiliki karakteristik kompleksitas waktu dan ruang yang berbeda sesuai dengan fungsi dan cara kerjanya.

Analisis kompleksitas dilakukan untuk mengetahui efisiensi sistem dalam menangani proses distribusi bantuan, pencarian rute, pengelolaan data lokasi, dan penyimpanan histori pengiriman. Kompleksitas dianalisis berdasarkan jumlah data lokasi, jumlah jalur distribusi, dan jumlah bantuan yang diproses.

4.1.1 Tabel Ringkasan

Tabel 4.1 Ringkasan Kompleksitas Algoritma

Struktur Data/Algoritma	Fungsi	Kompleksitas Waktu	Kompleksitas Ruang
<i>BST</i>	Insert	$O(\log n) - O(n)$	$O(n)$
<i>BST</i>	Search	$O(\log n) - O(n)$	$O(1)$
<i>Priority Queue</i>	Enqueue	$O(n)$	$O(n)$
<i>Priority Queue</i>	Dequeue	$O(1)$	$O(1)$
<i>Stack</i>	Push	$O(1)$	$O(1)$
<i>Stack</i>	Pop	$O(1)$	$O(1)$
<i>Graph</i>	BFS	$O(V + E)$	$O(V)$
<i>Dijkstra</i>	Shortest Path	$O(V^2 + E)$	$O(V)$

4.2 Perbandingan Teoritis dengan Eksperimen

Berdasarkan hasil pengujian program, kompleksitas teoritis yang diperoleh sesuai dengan perilaku sistem saat dijalankan. Operasi sederhana seperti push, pop, dan dequeue berjalan sangat cepat meskipun jumlah data bertambah karena memiliki kompleksitas konstan $O(1)$.

Pada BST, waktu pencarian lokasi meningkat secara bertahap ketika jumlah data lokasi bertambah. Namun, performa masih tergolong efisien selama struktur pohon tidak terlalu tidak seimbang.

Penggunaan Priority Queue menunjukkan bahwa proses penambahan bantuan membutuhkan waktu lebih lama dibanding penghapusan bantuan karena sistem harus mencari posisi prioritas yang sesuai sebelum data dimasukkan ke antrean.

Sementara itu, algoritma BFS dan Dijkstra membutuhkan waktu proses lebih besar dibanding struktur data lainnya karena harus menelusuri node dan jalur pada graph. Ketika jumlah lokasi dan rute distribusi meningkat, waktu eksekusi algoritma juga meningkat sesuai kompleksitas teoritisnya.

Secara keseluruhan, implementasi struktur data dan algoritma pada sistem telah mampu menangani simulasi distribusi bantuan dengan efisien untuk jumlah data menengah serta sesuai dengan analisis kompleksitas teoritis yang telah dilakukan.

BAB V

HASIL EKSPERIMEN

5.1 Tabel Runtime

Tabel 5.1 Hasil Runtime dengan Beragam Variasi

No	Deskripsi	Gambar
1	Variasi 100, 1000, 10000	<pre> ===== BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM ===== UKURAN STACK PUSH STACK POP PQ ENQ PQ DEQ BST INS BST SRC BFS DIJKSTRA 100 0.000031 0.000020 0.000185 0.000015 0.000082 0.000049 0.000075 0.000521 1000 0.000190 0.000151 0.012525 0.000125 0.001565 0.000081 0.000716 0.043140 10000 0.002919 0.001358 1.349223 0.001076 0.018047 0.000113 0.010194 4.828045 ===== Satuan runtime = detik ===== </pre>
2	Variasi 200, 2000, 20000	<pre> ===== BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM ===== UKURAN STACK PUSH STACK POP PQ ENQ PQ DEQ BST INS BST SRC BFS DIJKSTRA 200 0.000054 0.000032 0.000612 0.000045 0.000196 0.000067 0.000154 0.001805 2000 0.000496 0.000379 0.050795 0.000244 0.002415 0.000086 0.001583 0.177512 20000 0.004260 0.002988 5.116452 0.002330 0.035704 0.000129 0.023655 20.186837 ===== Satuan runtime = detik ===== </pre>
3	variasi 300, 3000, 30000	<pre> ===== BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM ===== UKURAN STACK PUSH STACK POP PQ ENQ PQ DEQ BST INS BST SRC BFS DIJKSTRA 300 0.000070 0.000048 0.001102 0.000033 0.000311 0.000065 0.000195 0.003652 3000 0.000640 0.000553 0.110159 0.000393 0.006950 0.000103 0.002583 0.421536 30000 0.008224 0.003974 11.857897 0.003622 0.058174 0.000163 0.040679 45.075083 ===== Satuan runtime = detik ===== </pre>

Berdasarkan hasil benchmark yang telah dilakukan, seluruh struktur data dan algoritma menunjukkan peningkatan runtime seiring bertambahnya ukuran data. Operasi pada Stack seperti push dan pop memiliki runtime paling kecil dan relatif stabil karena kedua operasi tersebut memiliki kompleksitas waktu $O(1)$.

Pada Priority Queue, proses enqueue mengalami peningkatan runtime yang sangat signifikan dibanding operasi lainnya. Hal ini terjadi karena proses penyisipan data harus melakukan traversal untuk menentukan posisi prioritas yang sesuai, sehingga kompleksitas

waktunya menjadi $O(n)$. Sementara itu, operasi dequeue tetap memiliki runtime kecil karena penghapusan dilakukan langsung pada elemen terdepan.

BST Insert juga mengalami peningkatan runtime ketika ukuran data bertambah, namun peningkatannya masih lebih kecil dibanding Priority Queue enqueue. Proses pencarian BST (search) menunjukkan performa yang cukup stabil dan efisien meskipun jumlah data meningkat.

Algoritma Dijkstra memiliki runtime terbesar pada ukuran data besar karena algoritma harus melakukan pencarian jalur terpendek pada graph dengan jumlah node dan edge yang lebih banyak. Hasil benchmark tersebut menunjukkan bahwa performa sistem secara umum telah sesuai dengan prediksi kompleksitas Big-O pada masing-masing struktur data dan algoritma yang digunakan.

5.2 Diskusi Hasil Prediksi Big-O

Berdasarkan hasil eksperimen yang telah dilakukan, performa sistem menunjukkan hasil yang sesuai dengan prediksi kompleksitas Big-O pada setiap struktur data dan algoritma yang digunakan. Pada proses pencarian data lokasi menggunakan Binary Search Tree (BST), waktu eksekusi meningkat secara bertahap ketika jumlah data bertambah. Namun, peningkatan tersebut tidak terlalu signifikan sehingga proses pencarian masih dapat dilakukan dengan efisien. Hasil ini sesuai dengan teori kompleksitas BST yang memiliki rata-rata kompleksitas waktu sebesar $O(\log n)$.

Pada struktur data Priority Queue proses penambahan bantuan menunjukkan peningkatan runtime yang lebih besar dibandingkan operasi lainnya. Hal tersebut terjadi karena sistem harus melakukan traversal untuk menentukan posisi prioritas yang sesuai sebelum data dimasukkan ke dalam antrian. Kondisi tersebut sesuai dengan kompleksitas teoritis enqueue pada Priority Queue berbasis linked list, yaitu $O(n)$.

Operasi pada struktur data Stack, seperti push dan pop, menunjukkan waktu eksekusi yang relatif stabil meskipun jumlah data bertambah. Hal ini disebabkan operasi hanya dilakukan pada bagian teratas stack tanpa perlu menelusuri seluruh elemen data. Dengan demikian, hasil eksperimen sesuai dengan teori kompleksitas waktu $O(1)$ pada operasi Stack.

Sementara itu, algoritma BFS dan Dijkstra menunjukkan peningkatan waktu eksekusi yang paling besar dibandingkan struktur data lainnya. Hal ini terjadi karena kedua algoritma harus melakukan penelusuran terhadap node dan jalur pada graph distribusi. Semakin banyak

jumlah vertex dan edge yang diproses, semakin besar pula waktu komputasi yang dibutuhkan. Hasil pengujian tersebut sesuai dengan teori kompleksitas BFS sebesar $O(V + E)$ dan Dijkstra sebesar $O(V^2 + E)$.

Secara keseluruhan, hasil eksperimen menunjukkan bahwa implementasi struktur data dan algoritma pada sistem telah berjalan sesuai dengan prediksi teoritis Big-O. Perbedaan kecil pada hasil runtime dapat dipengaruhi oleh faktor perangkat keras, kondisi sistem operasi, serta proses internal interpreter Python saat program dijalankan.

BAB VI

JAWABAN PERTANYAAN ANALISIS

6.1 Pertanyaan Analisis

6.1.1 Analisis Priority Queue Bantuan

Pertanyaan

Priority Queue Bantuan: KRITIS selalu diproses duluan (enqueue $O(n)$). Dalam kondisi bencana besar, 200 permintaan bantuan masuk dalam 1 jam dengan 30% berlevel KRITIS. Hitung total operasi enqueue untuk 200 permintaan dalam skenario terburuk (semua KRITIS datang terakhir). Bandingkan dengan pendekatan 3-bucket Queue (satu per level) di mana SERVE selalu cek KRITIS dulu: hitung total operasi dan identifikasi trade-off.

Jawaban

Pada implementasi Priority Queue berbasis linkedlist, proses enqueue dilakukan dengan menelusuri antrean untuk menemukan posisi prioritas yang sesuai. Dalam kondisi terburuk, seluruh bantuan prioritas KRITIS datang terakhir sehingga sistem harus melakukan traversal terhadap hampir seluruh elemen antrean sebelum data dimasukkan.

Untuk 200 permintaan bantuan, jumlah operasi enqueue menjadi sangat besar karena setiap data baru membutuhkan proses pencarian posisi prioritas. Semakin banyak data yang masuk, semakin besar pula jumlah traversal yang dilakukan sistem. Kondisi ini menunjukkan bahwa kompleksitas $O(n)$ pada enqueue cukup mempengaruhi performa ketika jumlah permintaan bantuan meningkat drastis.

Pada pendekatan 3-bucket Queue, sistem menggunakan tiga antrean terpisah berdasarkan level bencana, yaitu KRITIS, SEDANG, dan RINGAN. Proses enqueue dapat dilakukan langsung pada bagian belakang antrean yang sesuai sehingga waktu proses menjadi jauh lebih cepat dibanding Priority Queue tunggal.

Meskipun pendekatan 3-bucket Queue lebih efisien dalam proses enqueue, sistem membutuhkan pengelolaan beberapa antrean sekaligus sehingga penggunaan memori dan kompleksitas implementasi menjadi sedikit lebih besar. Priority Queue tunggal lebih sederhana untuk diimplementasikan, sedangkan 3-bucket Queue lebih cocok digunakan pada kondisi distribusi bantuan berskala besar dengan jumlah permintaan tinggi.

6.1.2 Analisis Perubahan Jalur pada Dijkstra

Pertanyaan

Dijkstra dari DEPOT_0 menghasilkan rute terpendek ke semua lokasi. Namun dalam bencana, rute bisa putus (jalan rusak = hapus edge). Jika 5 dari 50 edge rusak setelah Dijkstra dijalankan, apakah hasil Dijkstra masih valid? Bagaimana sistem harus merespons: (a) jalankan ulang Dijkstra $O(V^2+E)$, atau (b) gunakan pendekatan incremental. Jelaskan kompleksitas masing-masing.

Jawaban

Hasil algoritma Dijkstra tidak lagi sepenuhnya valid apabila terjadi perubahan pada graph, seperti penghapusan edge akibat jalan rusak. Jalur terpendek yang sebelumnya ditemukan dapat berubah atau bahkan tidak dapat digunakan lagi karena beberapa koneksi antar lokasi telah terputus.

Pendekatan pertama adalah menjalankan ulang algoritma Dijkstra dari awal. Cara ini paling mudah diterapkan karena sistem cukup menghitung kembali seluruh jalur berdasarkan kondisi graph terbaru. Namun, pendekatan ini membutuhkan waktu komputasi lebih besar karena seluruh node dan edge harus diproses ulang.

Pendekatan kedua adalah menggunakan metode incremental, yaitu hanya memperbarui bagian graph yang terdampak oleh perubahan jalur. Cara ini lebih efisien ketika jumlah edge yang rusak relatif sedikit karena sistem tidak perlu menghitung ulang seluruh graph. Akan tetapi, implementasi incremental lebih kompleks dibanding menjalankan ulang algoritma secara penuh.

Dalam kasus hanya 5 dari 50 edge yang rusak, pendekatan incremental lebih efisien dari sisi performa. Namun, apabila perubahan graph terjadi dalam jumlah besar, menjalankan ulang Dijkstra menjadi pilihan yang lebih sederhana dan stabil.

6.1.3 Analisis Multi-Source BFS

Pertanyaan

BFS dari depot menghasilkan himpunan lokasi yang dapat dijangkau. Jika 10 dari 38 lokasi tidak terjangkau dari DEPOT_0 tetapi terjangkau dari DEPOT_1, sistem harus mengalokasikan pengiriman ke depot terdekat secara otomatis. Desain algoritma multi-source

BFS yang menentukan depot optimal untuk setiap lokasi. Analisis Big-O-nya dan bandingkan dengan menjalankan Dijkstra dari setiap depot secara terpisah.

Jawaban

Untuk mengatasi masalah tersebut, sistem dapat menggunakan algoritma multi-source BFS. Pada metode ini, seluruh depot dimasukkan ke dalam queue pada awal proses pencarian. BFS kemudian berjalan secara bersamaan dari setiap depot untuk menentukan depot mana yang paling dekat dan dapat menjangkau suatu lokasi.

Setiap lokasi yang pertama kali dikunjungi akan ditandai dengan depot asalnya. Dengan cara ini, sistem dapat menentukan depot optimal secara otomatis tanpa perlu menjalankan BFS secara terpisah untuk setiap depot.

Pendekatan multi-source BFS lebih efisien dibanding menjalankan Dijkstra dari masing-masing depot secara terpisah. BFS hanya melakukan traversal satu kali terhadap graph sehingga waktu proses menjadi lebih cepat dan penggunaan sumber daya lebih efisien.

Sementara itu, menjalankan Dijkstra dari setiap depot membutuhkan komputasi lebih besar karena proses pencarian jalur minimum dilakukan berulang kali. Oleh karena itu, multi-source BFS lebih cocok digunakan untuk menentukan jangkauan depot pada sistem distribusi bantuan berskala besar.

6.1.4 Analisis Stack Log Pengiriman

Pertanyaan

Stack Log Pengiriman menyimpan semua transaksi bantuan (LIFO). Untuk audit pasca-bencana, dibutuhkan laporan kronologis (FIFO). Jelaskan dua cara membalik urutan Stack: (a) menggunakan Queue tambahan $O(n)$, (b) menggunakan Stack kedua $O(n)$. Implementasikan salah satunya dan analisis trade-off memori. Apakah ada cara $O(1)$ space untuk mengakses elemen pertama Stack?

Jawaban

Karena Stack bekerja menggunakan prinsip Last In First Out (LIFO), data terbaru selalu berada di bagian teratas. Kondisi ini kurang sesuai untuk kebutuhan audit kronologis yang memerlukan urutan First In First Out (FIFO).

Cara pertama untuk membalik urutan data adalah menggunakan Queue tambahan. Seluruh elemen pada Stack dipindahkan ke Queue sehingga urutan data menjadi kronologis. Pendekatan ini cukup mudah dipahami, tetapi membutuhkan memori tambahan untuk menyimpan seluruh elemen sementara.

Cara kedua adalah menggunakan Stack kedua. Data dari Stack utama dipindahkan satu per satu ke Stack baru sehingga urutan elemen menjadi terbalik. Metode ini juga membutuhkan memori tambahan, namun implementasinya lebih sederhana karena masih menggunakan struktur data yang sama.

Kedua pendekatan tersebut memiliki kompleksitas waktu linear karena seluruh elemen harus dipindahkan. Dari sisi memori, keduanya sama-sama membutuhkan ruang tambahan sebesar jumlah elemen yang diproses.

Tidak terdapat cara langsung untuk mengakses elemen pertama pada Stack dengan kompleksitas ruang konstan karena struktur Stack hanya menyediakan akses langsung pada elemen teratas. Untuk melihat elemen paling awal, sistem tetap harus melakukan traversal atau pemindahan data.

6.1.5 Analisis Sorting Laporan Bencana

Pertanyaan

Laporan Bencana memerlukan sorting lokasi berdasarkan tingkat urgensi ($\text{skor} = (4\text{-level}) * \text{populasi}$). Implementasikan Quick Sort pada Linked List untuk mengurutkan 38 lokasi berdasarkan skor ini. Bandingkan Quick Sort ($\text{avg } O(n \log n)$, $\text{worst } O(n^2)$) dengan Merge Sort ($O(n \log n)$ guaranteed) pada Linked List dari aspek: implementasi, penggunaan memori tambahan, dan cocok tidaknya untuk data hampir terurut.

Jawaban

Quick Sort dapat digunakan untuk mengurutkan data lokasi berdasarkan skor urgensi yang dihitung dari tingkat bencana dan jumlah populasi. Pada kondisi rata-rata, Quick Sort memiliki performa yang cukup cepat dan efisien sehingga cocok digunakan untuk proses sorting data berukuran menengah.

Namun, Quick Sort memiliki kelemahan pada kondisi terburuk, terutama ketika data sudah hampir terurut. Dalam kondisi tersebut, performa algoritma dapat menurun secara signifikan karena pembagian partisi menjadi tidak seimbang.

Merge Sort memiliki performa yang lebih stabil karena kompleksitas waktunya tetap konsisten pada berbagai kondisi data. Selain itu, Merge Sort lebih cocok diterapkan pada Linked List karena proses pembagian dan penggabungan node dapat dilakukan tanpa banyak perpindahan data.

Dari sisi implementasi, Quick Sort pada Linked List cenderung lebih sulit karena proses partisi node lebih kompleks dibanding array. Merge Sort lebih sederhana untuk diterapkan pada Linked List dan lebih aman digunakan ketika data sering berubah atau hampir terurut.

Berdasarkan karakteristik tersebut, Merge Sort lebih cocok digunakan pada sistem laporan bencana karena memberikan performa yang lebih stabil dan konsisten dibanding Quick Sort.

BAB VII

KESIMPULAN

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, sistem *Disaster Response Logistics System* berhasil dibangun menggunakan kombinasi beberapa struktur data dan algoritma, yaitu Binary Search Tree (BST), Priority Queue, Stack, Graph, Breadth First Search (BFS), dan algoritma Dijkstra. Sistem mampu melakukan simulasi distribusi bantuan bencana secara terstruktur melalui antarmuka berbasis Command Line Interface (CLI).

BST berhasil digunakan untuk pengelolaan data lokasi bencana sehingga proses pencarian dan pembaruan data dapat dilakukan dengan efisien. Priority Queue mampu mengatur prioritas distribusi bantuan berdasarkan tingkat keparahan bencana sehingga lokasi dengan status KRITIS dapat diproses lebih dahulu. Selain itu, Stack digunakan untuk menyimpan histori pengiriman bantuan sehingga proses pencatatan logistik dapat dilakukan dengan baik.

Implementasi graph, BFS, dan algoritma Dijkstra juga berhasil membantu proses analisis jalur distribusi bantuan. Sistem dapat menentukan wilayah yang dapat dijangkau dari depot tertentu serta mencari rute distribusi tercepat menuju lokasi tujuan. Berdasarkan hasil eksperimen, performa sistem menunjukkan hasil yang sesuai dengan prediksi kompleksitas Big-O pada masing-masing struktur data dan algoritma.

Secara keseluruhan, sistem telah memenuhi tujuan utama proyek, yaitu membantu simulasi pengelolaan distribusi logistik bencana dengan memanfaatkan konsep struktur data dan algoritma secara terintegrasi.

7.2 Refleksi Trade-Off Desain

Dalam proses pengembangan sistem, terdapat beberapa trade-off desain yang mempengaruhi performa dan kompleksitas implementasi. Penggunaan Priority Queue berbasis linked list memberikan kemudahan implementasi dan pengelolaan prioritas bantuan, namun proses enqueue memiliki kompleksitas waktu yang lebih besar ketika jumlah data meningkat.

Pemilihan BST sebagai struktur penyimpanan lokasi juga memberikan proses pencarian yang cukup cepat pada kondisi rata-rata. Akan tetapi, performa BST dapat menurun apabila struktur pohon menjadi tidak seimbang akibat urutan data tertentu.

Pada bagian pencarian rute, algoritma Dijkstra mampu menghasilkan jalur distribusi terpendek dengan akurat, tetapi membutuhkan waktu komputasi lebih besar dibanding algoritma traversal biasa seperti BFS. Sistem juga harus melakukan perhitungan ulang apabila terjadi perubahan jalur akibat kerusakan jalan atau bencana lanjutan.

Selain itu, penggunaan Stack untuk log pengiriman sangat efektif untuk proses rollback dan pencatatan transaksi terbaru. Namun, struktur LIFO kurang sesuai untuk kebutuhan audit kronologis sehingga dibutuhkan proses tambahan untuk menampilkan data dalam urutan FIFO.

Trade-off tersebut menunjukkan bahwa setiap struktur data dan algoritma memiliki kelebihan serta kekurangan masing-masing. Pemilihan struktur data dilakukan berdasarkan kebutuhan utama sistem, yaitu efisiensi distribusi bantuan dan kemudahan pengelolaan data logistik.

7.3 Saran Pengembangan

Sistem yang telah dibuat masih dapat dikembangkan lebih lanjut agar memiliki performa dan fitur yang lebih baik. Salah satu pengembangan yang dapat dilakukan adalah mengganti Priority Queue berbasis linked list dengan heap sehingga proses enqueue dan dequeue menjadi lebih efisien.

Pengembangan lain yang dapat dilakukan adalah penggunaan balanced tree seperti AVL Tree atau Red-Black Tree untuk menjaga performa pencarian data lokasi tetap stabil. Pada bagian graph, algoritma Dijkstra juga dapat dioptimalkan menggunakan priority heap agar waktu komputasi menjadi lebih cepat pada jumlah node yang besar.

Selain itu, sistem dapat dikembangkan menjadi aplikasi berbasis GUI atau web agar lebih mudah digunakan dibanding Command Line Interface. Integrasi dengan data peta digital dan sistem monitoring real-time juga dapat meningkatkan kemampuan sistem dalam menangani simulasi distribusi bantuan secara lebih realistis.

Dengan pengembangan lebih lanjut, sistem diharapkan dapat menjadi simulasi manajemen logistik bencana yang lebih efisien, akurat, dan mendekati implementasi pada kondisi nyata.

Lampiran

Lampiran A. Pembagian Tugas

No	Nama Anggota	Tugas
1	Raihan Sajidan	Implementasi Stack log pengiriman dan penyusunan presentasi (PPT)
2	Muhammad Ya'afi Daffa R.	Implementasi Graph dan algoritma Dijkstra, pengembangan main loop CLI, serta penyusunan laporan
3	Mohammad Nafif Bhanusetyo H.	Implementasi BST registry lokasi dan penyusunan presentasi (PPT)
4	Hafidh Aulia Dzakwan	Implementasi Priority Queue bantuan, penyusunan laporan, serta penyusunan presentasi (PPT)

Lampiran B. Tabel Runtime Lengkap

Variasi Ukuran Data 300, 3000, 30000

```
=====
BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM
=====
UKURAN   STACK PUSH   STACK POP   PQ ENQ      PQ DEQ      BST INS     BST SRC     DIJKSTRA
300       0.000096    0.000048   0.001088    0.000033    0.000283    0.000064    0.003636
3000      0.000653    0.000506   0.114017    0.000435    0.007014    0.000090    0.402569
30000     0.006894    0.004226   11.867916   0.003612    0.062840    0.000142    45.708443
=====
Satuan runtime = detik
=====
```

Variasi Ukuran Data 200, 2000, 20000

```
=====
BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM
=====
UKURAN   STACK PUSH   STACK POP   PQ ENQ      PQ DEQ      BST INS     BST SRC     DIJKSTRA
200       0.000120    0.000069   0.000511    0.000022    0.000176    0.000058    0.001633
2000      0.000488    0.000311   0.049674    0.000232    0.002486    0.000094    0.175031
20000     0.004138    0.002791   5.179466    0.002238    0.035918    0.000126    26.667840
=====
Satuan runtime = detik
=====
```

Variasi Ukuran Data 100, 1000, 10000

```
=====
BENCHMARK DISASTER RESPONSE LOGISTICS SYSTEM
=====
UKURAN    STACK PUSH    STACK POP    PQ ENQ      PQ DEQ      BST INS      BST SRC      DIJKSTRA
100       0.000066     0.000036     0.000166    0.000015    0.000082    0.000051    0.000514
1000      0.000196     0.000163     0.014134    0.000129    0.001281    0.000101    0.050523
10000     0.003385     0.001578     1.313803    0.001354    0.018274    0.000108    4.552433
=====
Satuan runtime = detik
=====
```

Lampiran C. Log Penggunaan AI

No	Aktivitas	Deskripsi
1	Penyusunan laporan	Membantu penyusunan struktur dan isi laporan BAB I sampai BAB VII
2	Analaisi Big-O	Membantu analisis kompleksitas waktu dan ruang pada struktur data dan algoritma
3	Penjelasan Kode program	Membantu penjelasan fungsi, modul, dan implementasi kode program
4	Penyusunan contoh input/output	Membantu pembuatan simulasi Command Line Interface (CLI)
5	Analisis eksperimen	Membantu interpretasi hasil runtime dan pengujian sistem
6	Penyusunan kode program	Membantu memberikan referensi implementasi struktur data dan algoritma pada program
7	Identifikasi eror program	Membantu menemukan dan menjelaskan error pada kode program selama proses pengembangan
8	Penggunaan Git dan GitHub	Membantu menjelaskan cara menggunakan Git untuk proses push, pull, commit, dan sinkronisasi repository